We create an object of the *Scan* class, and add to it all the columns that contain the specified *family* value. These values are then fetched into a ResultScanner object, which can be iterated to display the data.

Search

The Search() function can be used to search for a value; to call this function, you need to specify the resource name, family and *relationship*. This resembles an SQL *SELECT* statement with a *WHERE* clause.

```
Get g = new Get(Bytes.toBytes(resource));
Result r = htable.get(g);
byte [] value = r.getValue(Bytes.
toBytes(Family),
Bytes.toBytes(relation));
String valueStr = Bytes.toString(value);
```

First, we convert the *resource* to a byte stream, as in the case of *select()*. In the next statement, a Result type variable captures all the data records in the database corresponding to that resource. The *getValue()* function of the *Result* object extracts the value corresponding to that family and relationship. The next statement converts the byte stream into a printable string.

You can try using any of the above functions in your Java application.

Before you port to HBase

There are some pros and cons that you should be aware of if you're considering porting an application to use HBase.

A distributed database system stores records over different nodes instead of in a single central repository. Adding another node to a cluster is easy and cheap. If one of the nodes fails, the load is divided among the other nodes automatically, without affecting the whole system. It also allows for on-the-fly upgrades, even to the master server, without any downtime. In case the master itself fails, the failure is detected, and through master election and recovery, the last state is determined and the cluster continues running. The HBase integration with map/reduce makes it valuable and versatile. Every record has a time-stamp, so if a node is down, data is not lost and can be recovered.

The distributed database system does not allow us to choose the properties from the CAP theorem (a.k.a. Brewer's theorem—design systems can offer, at the most, two out of the three properties: Consistency, Availability, tolerance to network Partition). The HBase community has chosen CP.

It's a complex and very difficult task to fix things when they go wrong. It takes a skilled admin to go through the errors in the log file and troubleshoot. A small team may have a hard time figuring out bugs.

Normalisation is not possible in HBase. Another thing to note is that the primary key is the index key value, and no secondary key/foreign key is available. However, secondary keys can be emulated in the application. Things may improve in future versions.

Alternatives to HBase are Cassandra, Hypertable, Voldemort, CouchDB and MongoDB. Facebook and LinkedIn use Cassandra and Voldemort, which are somewhat similar to HBase. END

For further reading, consult these sites:

- http://wiki.apache.org/hadoop/
- http://search.iiit.ac.in/cloud/lectures.html
- http://jimbojw.com/wiki/index.php?title=Understanding_Hbase_and_BigTable

By: Amol Gupta and Harsh Vachhani

Amol is a tech freak, just out of college (JUIT), and is currently doing his internship at Infosys under the guidance of Harsh Vachhani. He has been an active member of the LUG at his college. You can follow him on Twitter (@amolgupta) or reach him at amolgupta1989@gmail.com. Harsh is currently working in the Education and Research Department of Infosys, Mysore. He is one of the founder members of the LUG at his alma mater, Jaypee University of Information Technology (JUIT). He frequently takes seminars on Open Source technologies for employees of Infosys. You can follow him on Twitter (@harshvachhani) or reach him at harshvachhani@gmail.com